

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS:

Total Marks: 30

Code of Conduct:

I POOJASREE B (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: POOJASREE B(192411091)

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate HTML5 semantic elements missing in the structure.
2. Modify the structure to make it standards-compliant.
3. Interpret the HTTP request generated during form submission.
4. Analyze the HTTP response cycle from server to browser.
5. Suggest improvements for usability and accessibility.

1. The given form does not contain semantic HTML5 elements that provide a proper page structure.

The following elements can be added:

- <header> – For the symposium title and introduction.
- <nav> – For navigation links.
- <main> – For the main registration content.
- <section> – For grouping related registration content.
- <footer> – For copyright and contact information.
- <fieldset> and <legend> – For grouping form controls.

These elements improve the structure, readability, and accessibility of the webpage.

2.

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>Symposium Registration</title>
</head>
```

```
<body>
```

```
<header>
```

```
<h1>College Online Symposium</h1>
</header>

<main>
  <section>
    <h2>Student Registration</h2>

    <form action="/register" method="post">

      <label for="name">Name:</label>
      <input type="text"
        id="name"
        name="name"
        required>

      <br><br>

      <label for="email">Email:</label>
      <input type="email"
        id="email"
        name="email"
        required>

      <br><br>

      <button type="submit">Submit</button>

    </form>
  </section>
</main>

<footer>
  <p>&copy; 2026 College Symposium</p>
</footer>

</body>

</html>
```

This structure is standards-compliant because it uses HTML5 doctype, semantic elements, proper labels, valid nesting, and required form validation.

3.

When the student enters the details and clicks **Submit**, the browser sends an HTTP POST request to the /register URL.

Example:

POST /register HTTP/1.1

Host: www.college.edu

Content-Type: application/x-www-form-urlencoded

name=Poojasree&email=poojasree@example.com

- POST is the HTTP method.
- /register is the server endpoint.
- Content-Type specifies the data format.
- The name and email are sent in the request body.

Thus, the browser sends the form data to the server for processing.

4.

The response cycle is:

Student fills the form



Clicks Submit



Browser sends HTTP POST request



Server receives the request



Server validates and processes the data



Server sends an HTTP response



Browser receives the response



Browser displays the result

A successful response may be:

HTTP/1.1 200 OK

Content-Type: text/html

The browser may then display:

Registration Successful

Common response codes include:

- 200 OK – Successful request.
- 400 Bad Request – Invalid data.
- 404 Not Found – Resource not found.
- 500 Internal Server Error – Server error.

5.

The following improvements can be made:

1. **Use proper labels** for all input fields.
2. **Use** required to prevent empty submissions.
3. **Use suitable input types**, such as email.
4. **Provide clear error messages** when invalid data is entered.
5. **Use descriptive button text**, such as Register Now.
6. **Ensure keyboard accessibility** using the Tab key.
7. **Use sufficient color contrast** for better readability.
8. **Make the webpage responsive** for mobile devices.
9. **Use semantic HTML5 elements** for screen-reader support.
10. **Use HTTPS** to protect submitted data during transmission.

These improvements make the symposium registration portal more **user-friendly, accessible, secure, and standards-compliant**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

Questions:

1. Identify improper nesting and missing closing tags.
2. Analyze how different browsers may interpret this structure.
3. Identify missing semantic elements (e.g., header, section).
4. Provide a corrected W3C-compliant HTML structure.
5. Suggest techniques to ensure cross-browser compatibility.

1. Identify improper nesting and missing closing tags.

The given HTML has the following problems:

```
<h1>Welcome
<p>About us
```

- The closing tag `</h1>` is missing.
- The closing tag `</p>` is missing.
- `<!DOCTYPE html>` is missing.
- The content is placed inside a generic `<div>` instead of semantic elements.

Correct form:

```
<h1>Welcome</h1>
<p>About us</p>
```

- 2.

Browsers try to automatically correct invalid HTML. However, different browsers may create different internal DOM structures when tags are not properly closed.

This can cause:

- Different page layouts.
- Inconsistent spacing and margins.
- Different CSS behavior.
- Unexpected nesting of elements.
- Different rendering in various browsers.

Also, because `<!DOCTYPE html>` is missing, some browsers may use **quirks mode**, which can cause further layout differences.

3.

The following semantic elements can be used:

- `<header>` – For the company heading and introductory content.
- `<nav>` – For navigation links.
- `<main>` – For the main page content.
- `<section>` – For grouping related content such as "About Us".
- `<footer>` – For copyright and contact information.

These elements make the webpage more meaningful and accessible.

4.

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
```

```
  <title>Company</title>
```

```
</head>
```

```
<body>
```

```
  <header>
```

```
    <h1>Welcome to Our Company</h1>
```

```
  </header>
```

```
  <main>
```

```
    <section>
```

```
      <h2>About Us</h2>
```

```
      <p>About us</p>
```

```
    </section>
```

```
  </main>
```



```
<footer>
  <p>&copy; 2026 Company</p>
</footer>
```

```
</body>
```

```
</html>
```

This structure is compliant because all elements are properly nested and closed, and HTML5 semantic elements are used.

5.

The following techniques can be used:

1. Use valid HTML and CSS.
2. Always include `<!DOCTYPE html>`.
3. Properly close and nest all HTML elements.
4. Use CSS resets or normalization to reduce browser differences.
5. Test the website in Chrome, Firefox, Edge, and Safari.
6. Use responsive design for different screen sizes.
7. Avoid browser-specific code whenever possible.
8. Use feature detection for modern web features.
9. Validate HTML and CSS using W3C validators.
10. Use progressive enhancement so the basic website works in all browsers.

These practices help produce a consistent layout and behavior across different browsers.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the form configuration.
2. Analyze why the HTTP request is not triggered.
3. Interpret the role of GET method in this scenario.
4. Propose a corrected implementation.
5. Suggest improvements for secure data transmission.

1.

The main issues are:

- The button is defined as:

```
<button type="button">Login</button>
```

It does not submit the form.

- The form uses the GET method for sending a password.
- With GET, the username and password may appear in the URL.

For example:

```
/submit?username=user&password=12345
```

This is not secure for login information.

2.

The button is:

```
<button type="button">Login</button>
```

A button with type="button" is a normal button and does not automatically submit the form.

Therefore:

User enters data



Clicks Login



Button action occurs



Form is not submitted



No HTTP request is generated

To submit the form, the button should be:

```
<button type="submit">Login</button>
```

3

The GET method sends form data as part of the URL.

For example:

```
/submit?username=pooja&password=12345
```

GET is generally used for retrieving data. It is not suitable for sending passwords because the data may be visible in:

- Browser history.
- Server logs.
- URLs.
- Proxy logs.

Therefore, GET should not be used for login credentials. POST should be used instead.

4.

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>Login</title>
```

```
</head>
```

```
<body>
```

```
  <form action="/submit" method="post">
```

```
    <label for="username">Username:</label>
```

```
    <input type="text"
```

```
id="username"
  name="username"
  required>

<br><br>

<label for="password">Password:</label>
<input type="password"
  id="password"
  name="password"
  required>

<br><br>

<button type="submit">Login</button>

</form>

</body>

</html>
```

Here:

- `method="post"` sends data in the request body.
- `type="submit"` triggers form submission.
- `required` prevents empty submission.
- `<label>` improves accessibility.

5.

The following improvements should be implemented:

1. **Use HTTPS** to encrypt communication between the browser and server.
2. **Use POST** instead of GET for login credentials.
3. **Validate input on the server side.**
4. **Never store passwords as plain text;** use secure password hashing.
5. **Use secure cookies** with Secure and HttpOnly attributes.
6. **Implement CSRF protection** for form requests.
7. **Avoid storing passwords in server logs.**
8. **Use rate limiting** to reduce brute-force login attempts.

Therefore, the corrected form should use **POST + HTTPS + server-side validation + secure password handling** for secure data transmission.